Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

За шестой семестр

**По дисциплине:** «Модели решения задач в интеллектуальных системах»

**Тема:** «Бинарная классификация с использованием кросс-энтропии (логистическая регрессия)»

**Выполнил:**
Студент 3 курса
Группы ИИ-26(1)
Прокопюк А.Д.
**Проверила:**
Андренко К.В.

Брест 2026

**Цель работы:** Изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провести **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).

**Задачи лабораторной работы:**

1. Модифицировать программу из ЛР №1:

- Заменить пороговую функцию активации на сигмоидную.
- Заменить функцию ошибки MSE на бинарную кросс-энтропию (BCE).
- Реализовать вычисление градиента ŷ-exi для обновления весов.

2. Реализовать обучение с **фиксированным** шагом (взять значение $\alpha$, показавшее наилучший результат в ЛР №1).

Запоминать суммарную ошибку Esp на каждой эпохе $p$.

3. Реализовать обучение с **адаптивным** шагом:

- На каждой итерации (после подачи одного образа) вычислять t (формула (2.36)).
- Использовать это $\alpha(t)$ для обновления весов и порога по градиенту BCE.
- Сохранять значения Esp для построения графика.

4. Провести сравнительный анализ:
    - Построить на одном графике кривые сходимости Esp для четырёх конфигураций:
        - MSE + фиксированный шаг (данные из ЛР №1);
        - MSE + адаптивный шаг (данные из ЛР №2);
        - BCE + фиксированный шаг;
        - BCE + адаптивный шаг.
    - Зафиксировать число эпох, необходимое для достижения заданной точности Es≤Ee (взять Ee из ЛР №2, например 0.01).
5. Визуализировать результаты:
    - Отобразить точки обучающей выборки (разными цветами для классов 0 и 1).
    - Построить разделяющую линию, полученную после обучения модели с BCE (адаптивный шаг).
    - Для сравнения также показать линию из ЛР №1 (если она отличается). **Описать** сравнение двух линий.
6. Реализовать режим функционирования:
    - Пользователь вводит координаты x1,x2.
    - Сеть вычисляет вероятность ŷ и, применяя порог, определяет предсказанный класс.
    - Точка отображается на графике с разделяющей линией (добавляется маркером с подписью класса или вероятности).
7. Сравнить скорость сходимости BCE и MSE, объяснить различия с позиции теории. Оценить влияние адаптивного шага на обучение с BCE. Сравнить итоговые разделяющие линии, полученные разными методами, и объяснить возможные расхождения.
8. Написать вывод о применимости разных функций потерь для задачи бинарной классификации.

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

class Perceptron:
    def __init__(self, input_size=0, learning_rate=0.1, target_accuracy=1e-2):
        self.X = np.array([])
        self.w = np.random.uniform(-1.0, 1.0, input_size + 1)
        self.learning_rate = learning_rate
        self.target = np.array([])
        self.target_accuracy = target_accuracy

    def set_X(self, X: np.array) -> None:
        X = np.array(X)

        if X.ndim == 1:
            X = X.reshape(1, -1)
        elif X.ndim != 2:
            print("X must be 1D or 2D array")
            return

        self.X = X
        self.X = np.insert(self.X, 0, -1, axis=1)

    def set_target(self, target: np.array) -> None:
        if self.X.ndim != 2:
            print("X not setted!")
            return

        if len(target) != len(self.X):
            print(f"Invalid size of target vector. It must have length =
{len(self.X)}. Now = {len(target)}")
            return

        self.target = target

    def get_wsum(self, X: np.array) -> np.array:
        return np.dot(X, self.w)

    def activate(self, arr_wsum: np.array) -> np.array:
        act = 1 / (1 + np.exp(-arr_wsum))
        return np.where(act > 0.5, 1, 0)

    # def sigm_activate(self, arr_wsum: np.array) -> np.array:
    #     #act =
    #     return 1 / (1 + np.exp(-arr_wsum))

    def prediction(self, X_input=None) -> np.array:
        X = self.X if X_input is None else X_input

        if X.size == 0:
            print("Input X vector not set!")
            return None

        wsum = self.get_wsum(X)
        y = 1 / (1 + np.exp(-wsum))

        return y
```

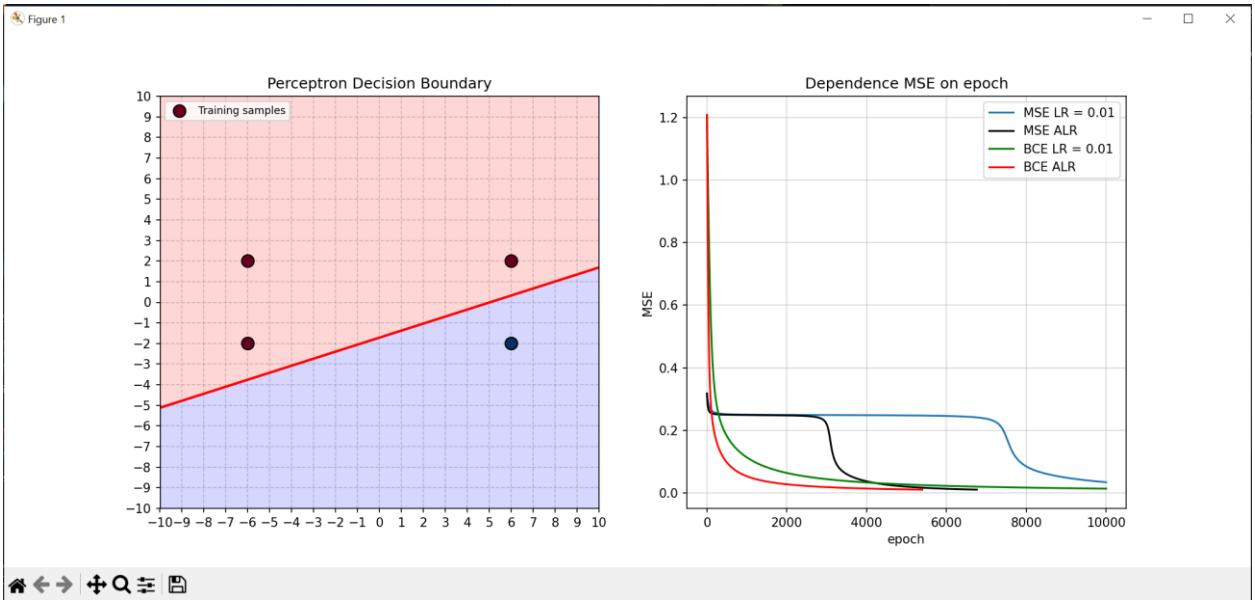
```
    def sigm_prediction(self, X_input = None) -> np.array:
        X = self.X if X_input is None else X_input

        wsum = self.get_wsum(X)
        #y = self.activate(wsum)
        y = 1 / (1 + np.exp(-wsum))

        return y

    def delta(self, y: np.array) -> None:
        error = y - self.target
        der = y * (1-y)
        self.w = self.w - self.learning_rate * np.dot(error * der, self.X) /
len(error)

    def bce_delta(self, y: np.array) -> None:
        error = y - self.target
        self.w = self.w - self.learning_rate * np.dot(error, self.X) / len(error)

    def mse(self, y: np.array) -> float:
        return np.mean((y - self.target) ** 2)

    def bce(self, y:np.array) -> float:
        return -np.mean(self.target * np.log(y) + (1 - self.target) * np.log(1 -
y))

    def train(self, epochs=500) -> np.array:   # поезд
        mse_history = []

        for epoch in range(epochs):
            y = self.sigm_prediction(self.X)

            mse = self.mse(y)
            mse_history.append(mse)

            if mse <= self.target_accuracy:
                print(f"Final for [LR{self.learning_rate}]:\nEpoch:
{epoch}\nMSE:{mse:.8f}")
                break

            # if len(mse_history) > 1 and abs(mse_history[-1] - mse_history[-2])
< self.target_accuracy:
            #     print(f"[LR{self.learning_rate}]Stopped on {epoch}: no
progress")
            #     break

            self.delta(y)

        return mse_history

    def train_adptive_lr(self, epochs=500) -> np.array:   # поезд
        mse_history = []
        self.learning_rate = 1 / np.mean(np.sum(self.X**2, axis=1))

        for epoch in range(epochs):
            y = self.sigm_prediction(self.X)

            mse = self.mse(y)
            mse_history.append(mse)
```

```
            if mse <= self.target_accuracy:
                print(f"Final for [ALR] ({self.learning_rate}):\nEpoch:
{epoch}\nMSE:{mse:.8f}")
                break

            # if len(mse_history) > 1 and abs(mse_history[-1] - mse_history[-2])
< self.target_accuracy:
            #     print(f"[ALR] Stopped on {epoch}: no progress")
            #     break

            self.delta(y)

        return mse_history

    def bce_train(self, epochs = 500):
        bce_history = []
        for epoch in range(epochs):
            y = self.sigm_prediction(self.X)

            bce = self.bce(y)
            bce_history.append(bce)

            if bce <= self.target_accuracy:
                print(f"[BCE fixed] Epoch {epoch}, BCE: {bce}, LR:
{self.learning_rate}")
                break

            self.bce_delta(y)
        return bce_history

    def bce_train_adaptive(self, epochs = 500):
        bce_history = []
        self.learning_rate = 1 / np.mean(np.sum(self.X**2, axis=1))
        for epoch in range(epochs):
            y = self.sigm_prediction(self.X)

            bce = self.bce(y)
            bce_history.append(bce)

            if bce <= self.target_accuracy:
                print(f"[BCE Adaptive] Epoch {epoch}, BCE: {bce}")
                break

            self.bce_delta(y)
        return bce_history

X_train = np.array([[6, 2], [-6, 2], [6, -2], [-6, -2]])
Y_targets = np.array([0, 0, 1, 0])
initial_weights = np.random.uniform(0, 1.0, 3)

plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 2)

test_p = Perceptron(input_size=2, learning_rate=0.01)
test_p.w = initial_weights.copy()
test_p.set_X(X_train)
test_p.set_target(Y_targets)
history = test_p.train(epochs=10000)
```

```
plt.plot(history, label=f'MSE LR = {0.01}')

adapt_p = Perceptron(input_size=2)
adapt_p.w = initial_weights.copy()
adapt_p.set_X(X_train)
adapt_p.set_target(Y_targets)
adapt_history = adapt_p.train_adptive_lr(epochs=10000)
plt.plot(adapt_history, label = f'MSE ALR', color='black')

bce_p = Perceptron(input_size=2, learning_rate=0.01)
bce_p.w = initial_weights.copy()
bce_p.set_X(X_train)
bce_p.set_target(Y_targets)
bce_history = bce_p.bce_train(epochs=10000)
plt.plot(bce_history, label = f'BCE LR = {0.01}', color = 'green')

a_bce_p = Perceptron(input_size=2, learning_rate=0.01)
a_bce_p.w = initial_weights.copy()
a_bce_p.set_X(X_train)
a_bce_p.set_target(Y_targets)
a_bce_history = a_bce_p.bce_train_adaptive(epochs=10000)
plt.plot(a_bce_history, label = f'BCE ALR', color = 'red')

plt.title("Dependence MSE on epoch")
plt.xlabel("epoch")
plt.ylabel("MSE")
plt.grid(True, which="both", ls="-", alpha=0.5)
plt.legend()
plt.grid(True)

p = Perceptron(input_size=2, learning_rate=0.001)
p.set_X(X_train)
p.set_target(Y_targets)
p.bce_train_adaptive(epochs=5000)

def plot_current_state(user_point=None, user_class=None):
    plt.subplot(1, 2, 1)

    plt.xticks(np.arange(-10, 11, 1))
    plt.yticks(np.arange(-10, 11, 1))

    plt.xlim([-10, 10])
    plt.ylim([-10, 10])

    plt.grid(True, linestyle='--', alpha=0.4, color='gray')

    xx, yy = np.meshgrid(np.linspace(-10, 10, 200), np.linspace(-10, 10, 200))
    grid_points = np.c_[np.ones(xx.ravel().shape) * -1, xx.ravel(), yy.ravel()]

    Z_linear = p.get_wsum(grid_points).reshape(xx.shape)
    Z_class = p.prediction(grid_points).reshape(xx.shape)

    plt.contourf(xx, yy, Z_class, levels=[-0.1, 0.5, 1.1], colors=["#ffcccc",
"#ccccff"], alpha=0.8)
    plt.contour(xx, yy, Z_linear, levels=[0], colors='red', linewidths=2)

    plt.scatter(X_train[:, 0], X_train[:, 1], c=Y_targets, cmap='RdBu',
                edgecolors='k', s=100, label='Training samples', zorder=5)
```

```
    if user_point is not None:
        plt.scatter(user_point[0], user_point[1], color='yellow', marker='*',
                    s=250, edgecolors='black', label=f'Pred: {user_class}',
zorder=6)

    plt.legend(loc='upper left', fontsize='small')
    plt.title("Perceptron Decision Boundary")

plot_current_state()
plt.show(block=False)

print("\nInput coordinates (x1, x2) from -7 to 7")
try:
    while True:
        line = input("Input x1 x2 (or 'exit'): ")
        if line.lower() == 'exit': break

        parts = line.split()
        if len(parts) != 2: continue

        coords = [float(p) for p in parts]
        user_x = np.array([[-1, coords[0], coords[1]]])

        pred = p.sigm_prediction(user_x)[0]
        pred_class = p.activate(p.get_wsum(user_x))[0]

        print(f"Result: {pred} (Class {p.activate(pred_class)})")

        plt.subplot(1, 2, 1)
        plt.cla()
        plot_current_state(coords, str(p.activate(pred_class)))
        plt.draw()
        plt.pause(0.1)
except ValueError:
    print("Invalid input.")

plt.show()
```

Результат:



Как мы можем видеть BCE справляется лучше MSE на сигмоидальной функции. На графиках MSE мы видим горб/ступеньку – он образуется из-за плато образующегося при использовании сигмоидной функции (при S = 0 y = 0.5 – здесь у нее низкая чувствительность из-за производной), как только шаг преодолевает ее, происходит резкий скачок. В BCE же производная не используется поэтому таких плато нет

Вывод: Изучил принципы бинарной классификации с вероятностной точки зрения. Реализовал однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провел **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).